

# .NET 期中作业：自定义读写锁

学号：2352038 姓名：陈子昂

## 一、要求

1. 并发读：多个线程可同时持有读锁；通过 `_readCount` 统计当前读者数量。
2. 多写者请求：多个线程可依次申请写锁；同一时刻最多一个写者处于临界区（`_isWriting`）。
3. 无写饥饿：新读者在存在等待写者（`_waitingWriters > 0`）时不再进入，写者优先获得后续机会。
4. 同步原语：仅使用 `lock`（等价于 `Monitor.Enter/Exit`）与 `Monitor.Wait` / `Monitor.PulseAll`，未使用更高级读写锁类型。

## 二、核心数据结构

- `_syncRoot`：公共监视器对象，保护所有状态变更与等待队列。
- `_readCount`：当前处于读临界区内的线程数量。
- `_isWriting`：是否有线程正持有写锁。
- `_waitingWriters`：已调用 `EnterWriteLock` 但尚未真正获得写锁的线程数（用于读侧让路，防止写饥饿）。

## 三、读锁工作流程

### 3.1 进入读锁 EnterReadLock

1. 在 `_syncRoot` 上获取互斥（`lock`）。
2. 循环等待，直到条件满足：当前没有写者（`!_isWriting`）且没有等待中的写者（`_waitingWriters == 0`）。  
否则调用 `Monitor.Wait(_syncRoot)` 释放锁并阻塞，被唤醒后重新检查条件。
3. 条件满足后，将 `_readCount` 加 1，表示增加一名读者。
4. 释放 `_syncRoot`，读者进入读临界区。

要点：一旦出现等待写者，新读者会阻塞在步骤 2，从而避免写者长期得不到调度（写饥饿）

## 3.2 退出读锁 ExitReadLock

1. 在 `_syncRoot` 上获取互斥。
2. 将 `_readCount` 减 1。
3. 若 `_readCount == 0`（已没有读者），调用 `Monitor.PulseAll(_syncRoot)`，唤醒可能正在等待的写者或其他读者，使其重新竞争检查条件。
4. 释放 `_syncRoot`。

## 四、写锁工作流程

### 4.1 进入写锁 EnterWriteLock

1. 在 `_syncRoot` 上获取互斥。
2. 将 `_waitingWriters` 加 1（声明“有写者在排队”，新读者将在 `EnterReadLock` 中被拦住）。
3. 循环等待，直到 没有读者（`_readCount == 0`）且没有其它写者占用（`!_isWriting`）；否则 `Monitor.Wait(_syncRoot)`。
4. 条件满足后，将 `_waitingWriters` 减 1（本线程不再算“排队中”），并置 `_isWriting = true`。
5. 释放 `_syncRoot`，写者进入写临界区。

### 4.2 退出写锁 ExitWriteLock

1. 在 `_syncRoot` 上获取互斥。
2. 将 `_isWriting` 置为 `false`。
3. 调用 `Monitor.PulseAll(_syncRoot)`，唤醒所有等待线程，由它们各自重新判断能否读或写。
4. 释放 `_syncRoot`。

## 五、测试用例设计

- 读者数量：3 个线程，每个线程执行 5 轮读操作。
- 写者数量：2 个线程，每个线程执行 5 轮写操作。
- 共享变量：整型 `_shareData`，写者每次加 1；读者打印当前读到的值。
- 时序：读、写循环内使用 `Thread.Sleep` 模拟临界区耗时，便于在控制台观察交错执行。

- **结束条件：**主线程对所有工作线程 `Join()`，全部结束后打印结束标记。

预期现象：可看到多个读线程在无写、无等待写者时连续打印同一数值（并发读）；写线程执行后数值递增；当写者在等时，不应再出现新读者在写者之前“插队”长期占用（体现写优先策略）。

## 六、测试结果

在项目目录执行 `dotnet run`（需先安装 .NET SDK）。以下为一次实际运行控制台摘录（顺序因调度可能略有不同，但应出现开始/结束标记及读写交替逻辑）：

```
== 自定义读写锁测试开始 ===
【读线程3】 读取值: 0
【读线程1】 读取值: 0
【读线程2】 读取值: 0
【写线程1】 写入新值: 1
【写线程2】 写入新值: 2
【读线程1】 读取值: 2
【读线程2】 读取值: 2
【读线程3】 读取值: 2
【写线程1】 写入新值: 3
【写线程2】 写入新值: 4
【读线程1】 读取值: 4
【读线程3】 读取值: 4
【读线程2】 读取值: 4
【写线程1】 写入新值: 5
【写线程2】 写入新值: 6
【读线程3】 读取值: 6
【读线程2】 读取值: 6
【读线程1】 读取值: 6
【写线程1】 写入新值: 7
【写线程2】 写入新值: 8
【读线程3】 读取值: 8
【读线程1】 读取值: 8
【读线程2】 读取值: 8
【写线程1】 写入新值: 9
【写线程2】 写入新值: 10
== 自定义读写锁测试结束 ===
```

**说明：**三个读线程各 5 次读、两个写线程各 5 次写，写操作共 10 次，故最终共享变量可达到 10；读者读到的值应与当时写进度一致，且无读写同时进入临界区的情况（由锁保证）。